

# Assessment Tool - 1

CO : 4

Name: Manoj M

Reg no: 192511060

course name: data structures

course code: CS40306

## 1) Linear Probing vs separate chaining

### Introduction:

Hashing stores data in a hash table using a hash function that maps keys to the index. When two or more keys produce the same index, a collision occurs.

### Comparative Analysis

| Parameter          | Linear Probing                  | separate chaining         |
|--------------------|---------------------------------|---------------------------|
| collision handling | searches for empty slot         | stores collided elements. |
| clustering         | suffers from primary clustering | No primary clustering     |
| Memory             | less memory                     | more memory for nodes     |
| Average time       | $O(1)$                          | $O(1)$                    |
| worst-case time    | $O(n)$                          | $O(n)$                    |
| deletion           | more complicated                | easier                    |

## operation and complexity:

| operation | case    | Linear Probing | separating chaining | on nodes<br>ity,<br>ops. |
|-----------|---------|----------------|---------------------|--------------------------|
| search    | Average | $O(1)$         | $O(1)$              |                          |
|           | worst   | $O(n)$         | $O(n)$              |                          |
| insertion | Average | $O(1)$         | $O(1)$              |                          |
|           | worst   | $O(n)$         | $O(n)$              |                          |

## Advantages & Limitations:

### Linear Probing

- \* simple to implement
- \* uses less additional memory.
- \* good cache locality

### separate chaining

- \* Avoids primary clustering
- \* Handles collision flexibility.
- \* can operate at higher load factors.

## Limitations

- \* Requires additional memory
- \* pointer traversal can reduce cache efficiency
- \* long chain can lead to  $O(n)$  operations.
- \* Dynamic memory allocation may introduce overhead.

## 2) quick sort vs merge sort :

Intro :

To compare large linked list of 10 million nodes considering memory, cache behaviour, time complexity, and then explain how the choice changes for arrays.

| Parameter       | quick sort                             | Merge sort                                             |
|-----------------|----------------------------------------|--------------------------------------------------------|
| Time - Average  | $O(n \log n)$                          | $O(n \log n)$                                          |
| Time - worst    | $O(n^2)$                               | $O(n \log n)$                                          |
| Extra space     | $O(\log n)$ average function           | $O(n)$ for merging                                     |
| cache behaviour | Better for Arrays                      | Poorer for arrays, good sequential access for lists.   |
| stability       | usually not stable                     | stable                                                 |
| main Advantages | in-place and cache friendly for Arrays | Predictable $O(n \log n)$ , efficient for linked list. |
| main limitation | worst: $O(n^2)$                        | Require extra memory.                                  |



For 10 million linked list nodes

• merge sort is preferable

\* because linked do not efficient Random Access.

\* merging can be performed by changing links rather than moving elements.

\* It is  $O(n \log n)$  time.

\* It is suitable for very large linked list.

What About Arrays?

For Arrays the choice may change

\* quick sort is often preferred because it works in place and has good cache locality.

\* merge sort requires  $O(n)$  extra memory but provides  $O(n \log n)$  time and stability.

Conclusion

For a 10-million node linked list  $\Rightarrow$   
merge sort is preferable.

### 3) Quadratic Probing vs Double Hashing

Intro:

To compare quadratic probing and double hashing based on clustering, collision-resolution efficiency, and suitability for hash tables.

#### Comparison Table

| Parameter            | Quadratic Probing                                            | Double Hashing                        |
|----------------------|--------------------------------------------------------------|---------------------------------------|
| Collision Handling   | uses quadratic jumps<br>: $h(k) + 1^2, h(k) + 2^2 \dots$     | uses 2nd hash function for jumps      |
| Clustering           | Reduces primary clustering but can have secondary clustering | Minimizes both primary and secondary. |
| Collision resolution | Good                                                         | Better                                |
| Dense hash table     | Less suitable as load factor increases                       | More suitable                         |
| Average time         | $O(1)$                                                       | $O(1)$                                |
| Worst time           | $O(n)$                                                       | $O(n)$                                |
| Memory               | Low: No extra space                                          | Same                                  |
| Advantage            | Simple & reduces primary clustering                          | Better for distributed keys           |

## Advantages:

### Quadratic Probing

- + simple to implement
- + reduces primary clustering
- + requires no extra memory

### Double Hashing

- + provides better collision
- + reduces both primary and secondary clustering
- + performs better in dense tables.

## Limitations

### Quadratic Probing

- + secondary clustering can occur
- + performance decreases at high load factors

### Double Hashing

- + require two hash functions.
- + slightly more computation.

## Conclusion:

Double Hashing is generally preferable for dense hash table.



# 4. Best case vs worst - case time complexity of sorting algorithms.

**Intro:** To compare best-case and worst-case complexity based on practical significance and impact on algorithm.

## Comparison

| Parameters      | Best case                                | Worst case                               |
|-----------------|------------------------------------------|------------------------------------------|
| Definition      | Minimum time needed                      | Maximum time                             |
| Input condition | Most favorable input                     | Least favorable input                    |
| Performance     | Fastest possible                         | slowest possible                         |
| Practical use   | shows how well an algorithm can perform. | Helps assess scalability                 |
| Algorithm       | when input is usually favorable          | when consistent performance is required. |

## Example:

| Algorithm      | Best case     | Worst case    |
|----------------|---------------|---------------|
| Bubble sort    | $O(n)$        | $O(n^2)$      |
| Insertion sort | $O(n)$        | $O(n^2)$      |
| Merge sort     | $O(n \log n)$ | $O(n \log n)$ |
| Quick sort     | $O(n \log n)$ | $O(n^2)$      |



## Advantages

## Algorithm section.

ex

- quick sort:  $O(n \log n)$  best case but  $O(n^2)$  worst case  
→ depends on pivot selection.
- merge sort:  $O(n \log n)$  in both best & worst cases ⇒ predictable.

## Conclusion:

Best case complexity shows the algorithm's potential under favourable conditions.

## 5) Recursive vs iterative quick sort

### Intro:

To compare recursive & iterative implementation of quick sort based on space complexity / stack usage, ease of implementation.

| Parameters      | Recursive quick sort           | Iterative quick                               |
|-----------------|--------------------------------|-----------------------------------------------|
| stack usage     | uses function call stack       | uses an explicit stack.                       |
| space - Average | $O(\log n)$                    | $O(\log n)$                                   |
| space - worst   | $O(n)$                         | $O(n)$                                        |
| implementation  | simpler & easier               | more complex                                  |
| stack overflow  | Higher risk for deep recursion | Lower risk because stack is controlled harder |

## Advantage

### Recursive

- + simple and natural implementation
- + easy to understand
- + code is shorter

### Iterative

- + gives explicit control over stack usage.
- + Avoids excessive recursion function calls.

## Limitation

### Recursive

- + can cause stack overflow when recursion becomes very deep.
- + worst case:  $O(n)$

### Iterative:

- + requires manual stack management
- + code is more complicated

## Conclusion

Recursive Quick sort is preferred when simplicity and readability are important.